

Queue - Complete Notes

Simple revision notes written in the same way we discussed in chat.

This chapter explains Queue from zero to interview level with examples, diagrams, operations, complexity, and common mistakes.

1. Why was Queue created?

Queue was created to model real-life waiting lines. The first person who enters the line should be served first. This rule is called FIFO: First In, First Out.

Think about a ticket counter, a printer queue, a support queue, or an order processing system. The first item added is the first item removed.

Main idea
Queue = waiting line
FIFO = First In, First Out
Add at one end, remove from the other

2. What is Queue?

A Queue is a collection where insertion happens at the rear and removal happens at the front. It does not work like a list where you randomly jump to any element.

Simple picture:

```
Front -> 10 -> 20 -> 30 -> 40 <- Rear
```

```
poll() removes 10 first
```

Queue is about processing order. The idea is not 'which element is biggest' or 'which is at index 0', but 'who came first?'.

3. Real-life analogy

Imagine people standing in a line for movie tickets. The person who comes first gets the ticket first. You do not allow someone in the middle to jump ahead. That is exactly the queue behavior.

Easy memory
Queue = line of people
First person in line is first served
No random middle access

4. Queue interface

Queue is an interface in Java. It defines the behavior, not the storage. Common implementations are LinkedList, PriorityQueue and ArrayDeque.

Examples:

```

Queue<Integer> q = new LinkedList<>();
Queue<Integer> q = new PriorityQueue<>();
Queue<Integer> q = new ArrayDeque<>();

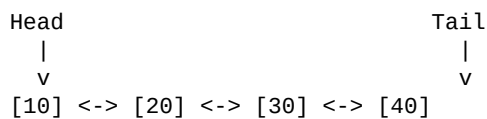
```

When you use the Queue interface, you are telling Java that you want queue behavior. The actual implementation can be different.

5. Why LinkedList is commonly used?

LinkedList is commonly used for Queue because it can add at the end and remove from the beginning in O(1). That matches queue behavior very well.

LinkedList queue idea:



No shifting is needed. Java simply changes the head or tail links when it adds or removes elements.

6. Queue operations

Method	Meaning
offer()	Add an element at the rear
poll()	Remove and return the front element
peek()	Return the front element without removing it
remove()	Remove the front element, but throw exception if empty
element()	Return front element, but throw exception if empty

7. offer()

offer() adds an element to the rear of the queue. If the queue has space, the element is inserted there. In LinkedList-based queue implementations, this is usually fast.

```

Queue<Integer> q = new LinkedList<>();
q.offer(10);
q.offer(20);
q.offer(30);

```

Result:
Front -> 10 -> 20 -> 30 <- Rear

8. poll()

poll() removes the front element and returns it. If the queue is empty, poll() returns null instead of throwing an exception.

Example:

Front -> 10 -> 20 -> 30

q.poll() returns 10

After poll:
Front -> 20 -> 30

poll() memory line

Removes from the front
Returns null if empty
Used when empty queue is possible

9. peek()

peek() looks at the front element without removing it. It is like checking who is first in line without letting them leave.

Example:

Front -> 20 -> 30

q.peek() returns 20

Queue stays the same

10. remove() and element()

remove() and element() are stricter versions. remove() is like poll(), but if the queue is empty it throws an exception. element() is like peek(), but if the queue is empty it throws an exception.

Quick comparison
poll() -> null if empty
peek() -> null if empty
remove() -> exception if empty
element() -> exception if empty

11. FIFO example

Example:

```
offer(10)
offer(20)
offer(30)
offer(40)
```

Queue:

Front -> 10 -> 20 -> 30 -> 40 <- Rear

poll() removes 10 first

That is FIFO: First In, First Out. The first element inserted is the first one removed.

12. Queue vs Stack

Queue	Stack
FIFO	LIFO
Front removed first	Top removed first
Line of people	Stack of plates

If the interviewer says FIFO, think Queue. If the interviewer says LIFO, think Stack.

13. Time complexity

Operation	Complexity	Why
offer()	O(1)	Add at rear
poll()	O(1)	Remove from front
peek()	O(1)	Read front
contains()	O(n)	Search node by node

14. Why Queue does not support random access

Queue is designed for processing in order. If you allow `get(index)` everywhere, the idea of a queue becomes less useful. So Queue focuses on front and rear operations rather than index-based operations.

Interview-friendly idea
Queue is a line, not a list with random access
You care about front and rear

15. Common mistakes

One mistake is thinking Queue can access the middle element directly. Another mistake is expecting Queue to always be sorted. Only PriorityQueue behaves based on priority; normal Queue follows FIFO.

A further mistake is using `remove()` or `element()` without checking whether the queue is empty.

16. Interview questions

What is Queue? Why is it FIFO? What is the difference between `offer()` and `add()`? What is the difference between `poll()` and `remove()`? What is the difference between `peek()` and `element()`? Why is LinkedList commonly used as a Queue?

17. Final revision sheet

Topic	Simple meaning
Queue	Waiting line
Rule	FIFO
offer()	Add at rear
poll()	Remove front, return null if empty
peek()	View front, return null if empty
remove()	Remove front, exception if empty
element()	View front, exception if empty
Best implementation	LinkedList is common

Best use	Order-based processing
----------	------------------------

One-line memory: Queue = FIFO. Add at rear, remove from front. poll() and peek() return null if empty.